Assignment 4

Ashmita Neupane

2025-04-10

```r
#Load Libraries
library(mlbench)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(performanceEstimation)
library(e1071)
library(randomForest)
```

```
## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
## Attaching package: 'randomForest'

## The following object is masked from 'package:dplyr':
##
##     combine
```

```r
library(nnet)
library(gbm)
```

```
## Loaded gbm 2.2.2

## This version of gbm is no longer under development. Consider transitioning to gbm3, https://github.c
```

```r
library(caret)
```

```
## Loading required package: ggplot2

##
## Attaching package: 'ggplot2'

## The following object is masked from 'package:randomForest':
##
##     margin

## Loading required package: lattice
```

```r
library(ggplot2)
```

```r
#Load the data
data("BreastCancer")
bc <- BreastCancer

#Check for missing values
colSums(is.na(BreastCancer))
```

```
##              Id    Cl.thickness       Cell.size      Cell.shape   Marg.adhesion
##               0               0               0               0               0
##    Epith.c.size     Bare.nuclei     Bl.cromatin Normal.nucleoli         Mitoses
##               0              16               0               0               0
##           Class
##               0
```

```r
#Convert Bare.nuclei from factor to numeric
bc$Bare.nuclei <- as.numeric(as.character(bc$Bare.nuclei))

#Replace NA with mean value
mean_val <- mean(bc$Bare.nuclei, na.rm = TRUE)
bc$Bare.nuclei[is.na(bc$Bare.nuclei)] <- mean_val

#Remove Id column, not useful for prediction
bc$Id <- NULL

#Convert all remaining factor variables except target variable - Class, to numeric
bc <- bc %>%
  mutate(across(where(is.factor) & !matches("Class"), ~ as.numeric(as.character(.))))

#Define predictive task for performance estimation
task <- PredTask(Class ~ ., bc, "BreastCancer", type = "class")
```

```r
#Define workflows for models
#Logistic Regression
mywf_glm <- function(form, train, test) {
  model <- glm(form, train, family = "binomial")
```

```r
  prob <- predict(model, test, type = "response")
  pred <- factor(ifelse(prob > 0.5, "malignant", "benign"),
                 levels = c("benign", "malignant"))
  list(trues = responseValues(form, test), preds = pred)
}
wf_glm <- workflowVariants(wf = "mywf_glm")

#Neural Network
mywf_nnet <- function(form, train, test, size = c(2)) {
  model <- nnet(form, train, size = size, trace = FALSE)
  prob <- predict(model, test, type = "raw")
  pred <- factor(ifelse(prob > 0.5, "malignant", "benign"),
                 levels = c("benign", "malignant"))
  list(trues = responseValues(form, test), preds = pred)
}
wf_nnet <- workflowVariants(wf = "mywf_nnet", size = 1:3)

#Random Forest
wf_rf <- workflowVariants(learner = "randomForest",
                          learner.pars = list(ntree = c(50)))

#Support Vector Machine
wf_svm <- workflowVariants(learner = "svm",
                           learner.pars = list(kernel = "radial"))

#Gradient boosting
mywf_gbm <- function(form, train, test, n.trees = 100) {
  train$ClassNum <- ifelse(train$Class == "malignant", 1, 0)
  test$ClassNum <- ifelse(test$Class == "malignant", 1, 0)

  model <- gbm(formula = ClassNum ~ ., distribution = "bernoulli",
               data = train[, -which(names(train) == "Class")],
               n.trees = n.trees, interaction.depth = 2,
               shrinkage = 0.1, verbose = FALSE)
  probs <- predict(model, newdata = test, n.trees = n.trees, type = "response")
  pred <- factor(ifelse(probs > 0.5, "malignant", "benign"),
                 levels = c("benign", "malignant"))
  list(trues = responseValues(form, test), preds = pred)
}
wf_gbm <- workflowVariants(wf = "mywf_gbm", n.trees = c(50, 100))
```

```r
#Set up Performance Estimation
results <- performanceEstimation(
  tasks = task,
  workflows = c(wf_glm, wf_nnet, wf_rf, wf_svm, wf_gbm),

  #10 folds CV with 2 repetitions
  estTask = EstimationTask(method = CV(nReps = 2, nFolds = 10, seed = 1234),
                           metrics = c("acc", "rec", "prec", "F"))
)
```

```
##
##
```

```
## ##### PERFORMANCE ESTIMATION USING  CROSS VALIDATION  #####
##
## ** PREDICTIVE TASK :: BreastCancer
##
## ++ MODEL/WORKFLOW :: mywf_glm.v1
## Task for estimating  acc,rec,prec,F  using
##  2 x 10 - Fold Cross Validation
##   Run with seed =  1234
## Iteration :********************
##
##
## ++ MODEL/WORKFLOW :: mywf_nnet.v1
## Task for estimating  acc,rec,prec,F  using
##  2 x 10 - Fold Cross Validation
##   Run with seed =  1234
## Iteration :********************
##
##
## ++ MODEL/WORKFLOW :: mywf_nnet.v2
## Task for estimating  acc,rec,prec,F  using
##  2 x 10 - Fold Cross Validation
##   Run with seed =  1234
## Iteration :********************
##
##
## ++ MODEL/WORKFLOW :: mywf_nnet.v3
## Task for estimating  acc,rec,prec,F  using
##  2 x 10 - Fold Cross Validation
##   Run with seed =  1234
## Iteration :********************
##
##
## ++ MODEL/WORKFLOW :: randomForest
## Task for estimating  acc,rec,prec,F  using
##  2 x 10 - Fold Cross Validation
##   Run with seed =  1234
## Iteration :********************
##
##
## ++ MODEL/WORKFLOW :: svm
## Task for estimating  acc,rec,prec,F  using
##  2 x 10 - Fold Cross Validation
##   Run with seed =  1234
## Iteration :********************
##
##
## ++ MODEL/WORKFLOW :: mywf_gbm.v1
## Task for estimating  acc,rec,prec,F  using
##  2 x 10 - Fold Cross Validation
##   Run with seed =  1234
## Iteration :********************
##
##
## ++ MODEL/WORKFLOW :: mywf_gbm.v2
```

```
## Task for estimating  acc,rec,prec,F  using
##  2 x 10 - Fold Cross Validation
##   Run with seed =  1234
## Iteration :********************
```

```r
summary(results)
```

```
##
## == Summary of a  Cross Validation Performance Estimation Experiment ==
##
## Task for estimating  acc,rec,prec,F  using
##  2 x 10 - Fold Cross Validation
##   Run with seed =  1234
##
## * Predictive Tasks ::  BreastCancer
## * Workflows  ::  mywf_glm.v1, mywf_nnet.v1, mywf_nnet.v2, mywf_nnet.v3, randomForest, svm, mywf_gbm.v
##
## -> Task:  BreastCancer
##   *Workflow: mywf_glm.v1
##                acc        rec       prec          F
## avg     0.96449275 0.97369409 0.97280012 0.97289739
## std     0.02021070 0.02071365 0.02732009 0.01557085
## med     0.97101449 0.97646733 0.97801932 0.97752525
## iqr     0.01811594 0.02724707 0.04278446 0.01708211
## min     0.91304348 0.92156863 0.89130435 0.93181818
## max     1.00000000 1.00000000 1.00000000 1.00000000
## invalid 0.00000000 0.00000000 0.00000000 0.00000000
##
##   *Workflow: mywf_nnet.v1
##                acc        rec       prec          F
## avg     0.95434783 0.95832126 0.97248393 0.96476205
## std     0.02268475 0.02920084 0.02999441 0.01766563
## med     0.95652174 0.96684211 0.97674419 0.96153214
## iqr     0.02898551 0.04291551 0.04020408 0.02109569
## min     0.89855072 0.89361702 0.87234043 0.92134831
## max     1.00000000 1.00000000 1.00000000 1.00000000
## invalid 0.00000000 0.00000000 0.00000000 0.00000000
##
##   *Workflow: mywf_nnet.v2
##               acc        rec      prec          F
## avg     0.8405797 0.97779598 0.8518228 0.89950161
## std     0.1615214 0.02694357 0.1705493 0.09707259
## med     0.9492754 1.00000000 0.9587585 0.96107332
## iqr     0.2934783 0.04552846 0.3224638 0.16745505
## min     0.5507246 0.93023256 0.5507246 0.71028037
## max     1.0000000 1.00000000 1.0000000 1.00000000
## invalid 0.0000000 0.00000000 0.0000000 0.00000000
##
##   *Workflow: mywf_nnet.v3
##                acc        rec       prec          F
## avg     0.95652174 0.96506015 0.96996325 0.96714775
## std     0.02443267 0.02608950 0.02568580 0.01780144
## med     0.94927536 0.96643394 0.97587209 0.96151955
## iqr     0.03260870 0.02304822 0.04552846 0.02630365
```

```
## min     0.92753623 0.90196078 0.91836735 0.94736842
## max     1.00000000 1.00000000 1.00000000 1.00000000
## invalid 0.00000000 0.00000000 0.00000000 0.00000000
##
##   *Workflow: randomForest
##                acc        rec       prec          F
## avg     0.96594203 0.97034277 0.97764265 0.97360423
## std     0.02219208 0.02389906 0.02805917 0.01755886
## med     0.97101449 0.97674419 0.98936170 0.97700846
## iqr     0.03260870 0.02374433 0.04104128 0.02804656
## min     0.92753623 0.91489362 0.91111111 0.93975904
## max     1.00000000 1.00000000 1.00000000 1.00000000
## invalid 0.00000000 0.00000000 0.00000000 0.00000000
##
##   *Workflow: svm
##                acc        rec       prec          F
## avg     0.96666667 0.96495855 0.98433084 0.97411909
## std     0.02054971 0.02938278 0.02206321 0.01607139
## med     0.97101449 0.96643394 0.99019608 0.97726098
## iqr     0.02898551 0.02884637 0.02222222 0.02366226
## min     0.92753623 0.89361702 0.91111111 0.94252874
## max     1.00000000 1.00000000 1.00000000 1.00000000
## invalid 0.00000000 0.00000000 0.00000000 0.00000000
##
##   *Workflow: mywf_gbm.v1
##                acc        rec       prec          F
## avg     0.96159420 0.97140433 0.97020695 0.97049590
## std     0.02219208 0.02186906 0.02677354 0.01697245
## med     0.96376812 0.97559524 0.97777778 0.97294372
## iqr     0.03260870 0.02900379 0.02917882 0.03017995
## min     0.91304348 0.93478261 0.91111111 0.94000000
## max     0.98550725 1.00000000 1.00000000 0.98947368
## invalid 0.00000000 0.00000000 0.00000000 0.00000000
##
##   *Workflow: mywf_gbm.v2
##                acc        rec       prec          F
## avg     0.96666667 0.97140433 0.97783592 0.97435425
## std     0.02210473 0.02186906 0.02462101 0.01698768
## med     0.97101449 0.97559524 0.97849214 0.97493734
## iqr     0.02898551 0.02900379 0.02935223 0.02355115
## min     0.92753623 0.93478261 0.91111111 0.94252874
## max     1.00000000 1.00000000 1.00000000 1.00000000
## invalid 0.00000000 0.00000000 0.00000000 0.00000000
```

```r
plot(results)
```

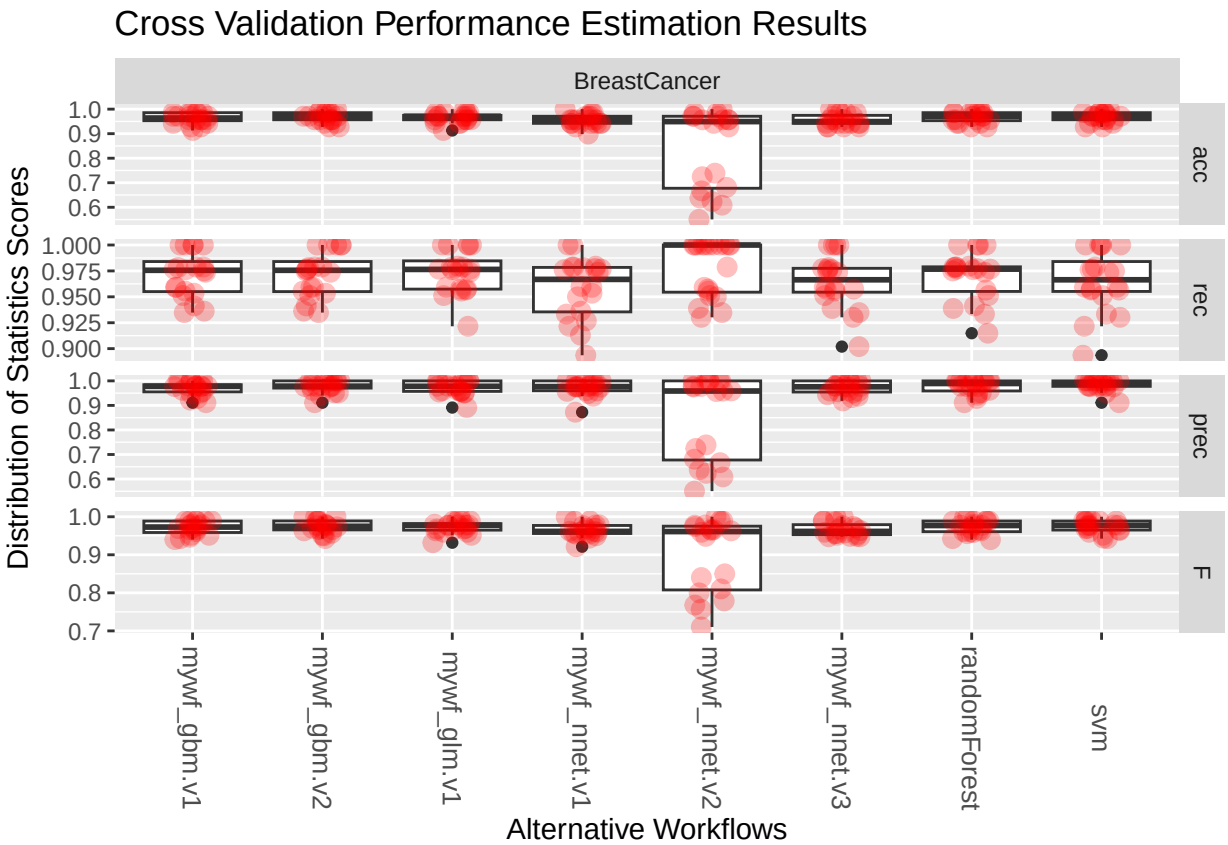
```r
#Create data frame from manually extracted summary result
model_metrics <- data.frame(
  Model = c(
    "Logistic Regression", "Neural Net (size=1)", "Neural Net (size=2)",
    "Neural Net (size=3)", "Random Forest", "SVM",
    "Gradient Boosting (50)", "Gradient Boosting (100)"
  ),
  Accuracy = c(0.964, 0.954, 0.841, 0.957, 0.966, 0.967, 0.962, 0.967),
  Precision = c(0.973, 0.972, 0.852, 0.970, 0.978, 0.984, 0.970, 0.978),
  Recall = c(0.974, 0.958, 0.978, 0.965, 0.970, 0.965, 0.971, 0.971),
  F1_Score = c(0.973, 0.965, 0.899, 0.967, 0.974, 0.974, 0.970, 0.974)
)

#Arrange in descending order
model_metrics <- model_metrics %>%
  arrange(desc(F1_Score))

#Print the summary table
print(model_metrics)
```

```
##                     Model Accuracy Precision Recall F1_Score
## 1           Random Forest    0.966     0.978  0.970    0.974
## 2                     SVM    0.967     0.984  0.965    0.974
## 3 Gradient Boosting (100)    0.967     0.978  0.971    0.974
## 4     Logistic Regression    0.964     0.973  0.974    0.973
## 5  Gradient Boosting (50)    0.962     0.970  0.971    0.970
```

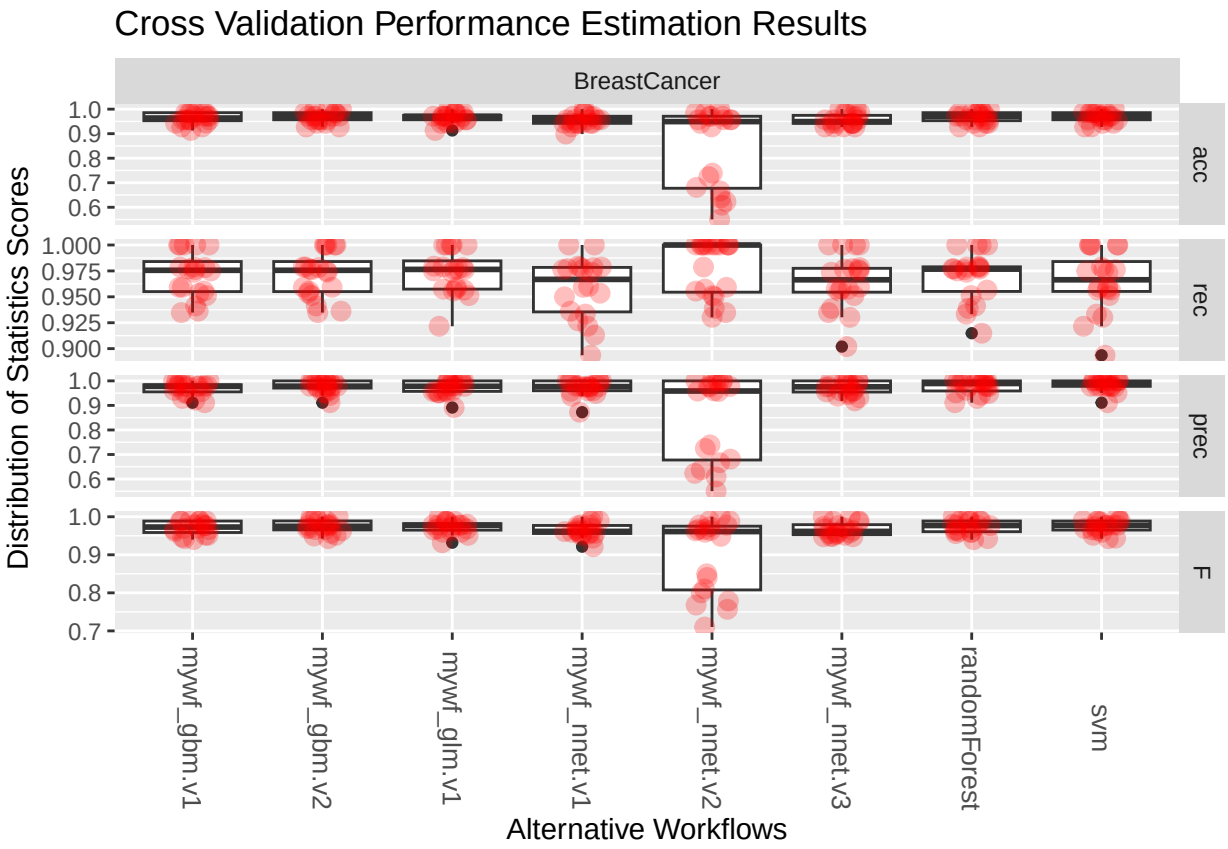
```
## 6      Neural Net (size=3)     0.957      0.970  0.965     0.967
## 7      Neural Net (size=1)     0.954      0.972  0.958     0.965
## 8      Neural Net (size=2)     0.841      0.852  0.978     0.899
```

```r
plot(results, metric = "acc")
```

```r
plot(results, metric = "F")
```




```
plot(results, metric = "prec")
```




```
plot(results, metric = "rec")
```

```r
topPerformers(results)
```

```
## $BreastCancer
##          Workflow Estimate
## acc  mywf_nnet.v2    0.841
## rec  mywf_nnet.v1    0.958
## prec mywf_nnet.v2    0.852
## F    mywf_nnet.v2      0.9
```

```r
rankWorkflows(results, top = 3)
```

```
## $BreastCancer
## $BreastCancer$acc
##       Workflow  Estimate
## 1 mywf_nnet.v2 0.8405797
## 2 mywf_nnet.v1 0.9543478
## 3 mywf_nnet.v3 0.9565217
##
## $BreastCancer$rec
##       Workflow  Estimate
## 1 mywf_nnet.v1 0.9583213
## 2          svm 0.9649585
## 3 mywf_nnet.v3 0.9650602
##
```

```
## $BreastCancer$prec
##       Workflow  Estimate
## 1 mywf_nnet.v2 0.8518228
## 2 mywf_nnet.v3 0.9699633
## 3  mywf_gbm.v1 0.9702069
##
## $BreastCancer$F
##       Workflow  Estimate
## 1 mywf_nnet.v2 0.8995016
## 2 mywf_nnet.v1 0.9647620
## 3 mywf_nnet.v3 0.9671477
```

```r
#Since RF, SVM, and GBM had same F1-Score the data was trained to get confusion
#matrix for all three models
set.seed(123)
#70% training 30% testing data
train_idx <- sample(1:nrow(bc), 0.7 * nrow(bc))
train <- bc[train_idx, ]
test <- bc[-train_idx, ]

#Convert target variable to numeric for GBM
train$ClassNum <- ifelse(train$Class == "malignant", 1, 0)
test$ClassNum <- ifelse(test$Class == "malignant", 1, 0)

#Train GBM model
gbm_model <- gbm(ClassNum ~ ., data = train[, -which(names(train) == "Class")],
                 distribution = "bernoulli",
                 n.trees = 100,
                 interaction.depth = 2,
                 shrinkage = 0.1,
                 verbose = FALSE)
#Predict on test set
probs <- predict(gbm_model, newdata = test, n.trees = 100, type = "response")
gbm_pred <- factor(ifelse(probs > 0.5, "malignant", "benign"),
                   levels = c("benign", "malignant"))

#Confusion matrix for GBM
confusionMatrix(gbm_pred, test$Class, positive = "malignant")
```

```
## Confusion Matrix and Statistics
##
##            Reference
## Prediction  benign malignant
##   benign       144         3
##   malignant      6        57
##
##                Accuracy : 0.9571
##                  95% CI : (0.9202, 0.9802)
##     No Information Rate : 0.7143
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.8966
##
##  Mcnemar's Test P-Value : 0.505
```

```
##
##             Sensitivity : 0.9500
##             Specificity : 0.9600
##          Pos Pred Value : 0.9048
##          Neg Pred Value : 0.9796
##              Prevalence : 0.2857
##          Detection Rate : 0.2714
##    Detection Prevalence : 0.3000
##       Balanced Accuracy : 0.9550
##
##        'Positive' Class : malignant
##
```

```r
table(train$Class)
```

```
##
##    benign malignant
##       308       181
```

```r
table(test$Class)
```

```
##
##    benign malignant
##       150        60
```

```r
table(gbm_pred)
```

```
## gbm_pred
##    benign malignant
##       147        63
```

```r
#Train SVM Model
svm_model <- svm(Class ~ ., data = train, kernel = "radial")

#Predict on test data
svm_pred <- predict(svm_model, test)

#Confusion matrix
confusionMatrix(svm_pred, test$Class, positive = "malignant")
```

```
## Confusion Matrix and Statistics
##
##            Reference
## Prediction  benign malignant
##   benign       150         0
##   malignant      0        60
##
##                Accuracy : 1
##                  95% CI : (0.9826, 1)
##     No Information Rate : 0.7143
##     P-Value [Acc > NIR] : < 2.2e-16
```

```
##
##                   Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
##             Sensitivity : 1.0000
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 1.0000
##              Prevalence : 0.2857
##          Detection Rate : 0.2857
##    Detection Prevalence : 0.2857
##       Balanced Accuracy : 1.0000
##
##        'Positive' Class : malignant
##
```

```r
table(train$Class)
```

```
##
##    benign malignant
##       308       181
```

```r
table(test$Class)
```

```
##
##    benign malignant
##       150        60
```

```r
table(svm_pred)
```

```
## svm_pred
##    benign malignant
##       150        60
```

```r
#Train Random Forest model
rf_model <- randomForest(Class ~ ., data = train, ntree = 50)

#Predict on test data
rf_pred <- predict(rf_model, test)

#Confusion Matrix
confusionMatrix(rf_pred, test$Class, positive = "malignant")
```

```
## Confusion Matrix and Statistics
##
##            Reference
## Prediction  benign malignant
##   benign       150         0
##   malignant      0        60
##
```

```
##                Accuracy : 1
##                  95% CI : (0.9826, 1)
##     No Information Rate : 0.7143
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
##             Sensitivity : 1.0000
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 1.0000
##              Prevalence : 0.2857
##          Detection Rate : 0.2857
##    Detection Prevalence : 0.2857
##       Balanced Accuracy : 1.0000
##
##        'Positive' Class : malignant
##
```

```r
table(train$Class)
```

```
##
##    benign malignant
##       308       181
```

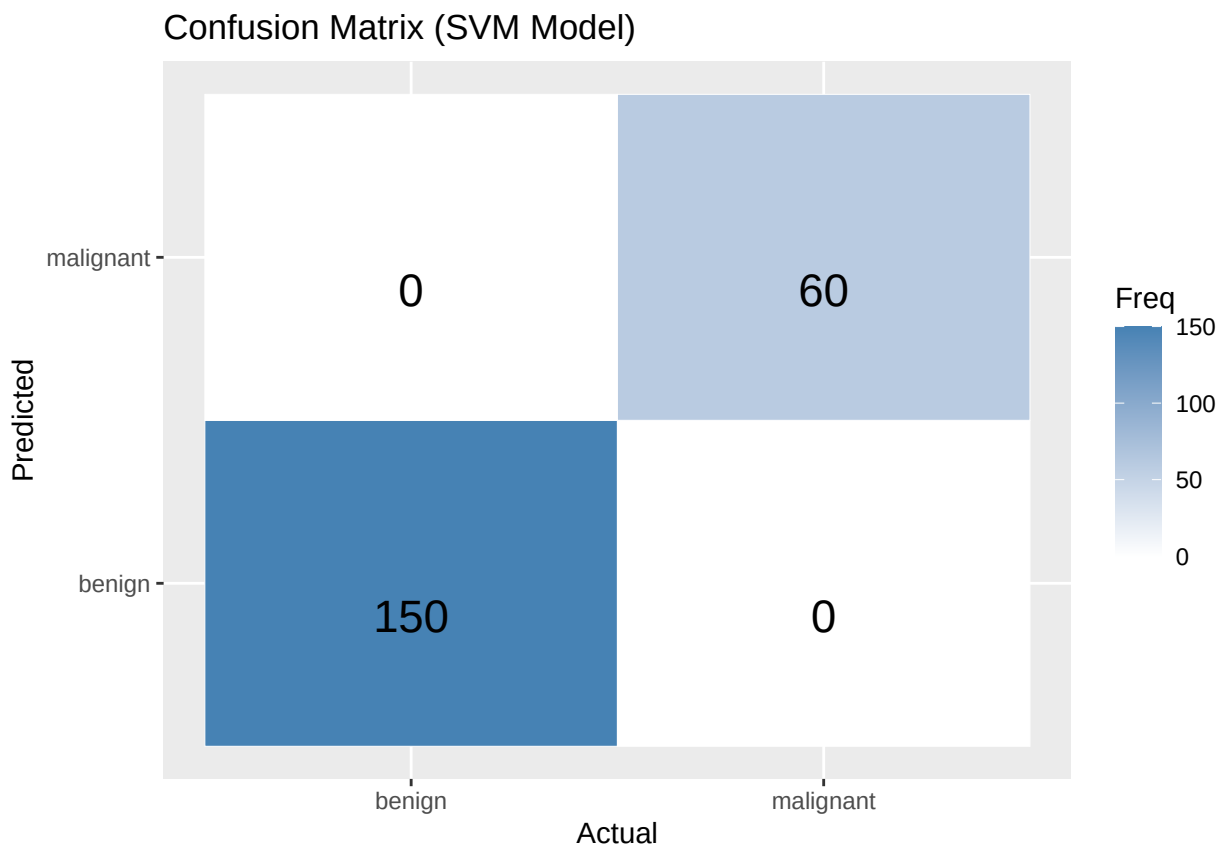
```r
table(test$Class)
```

```
##
##    benign malignant
##       150        60
```

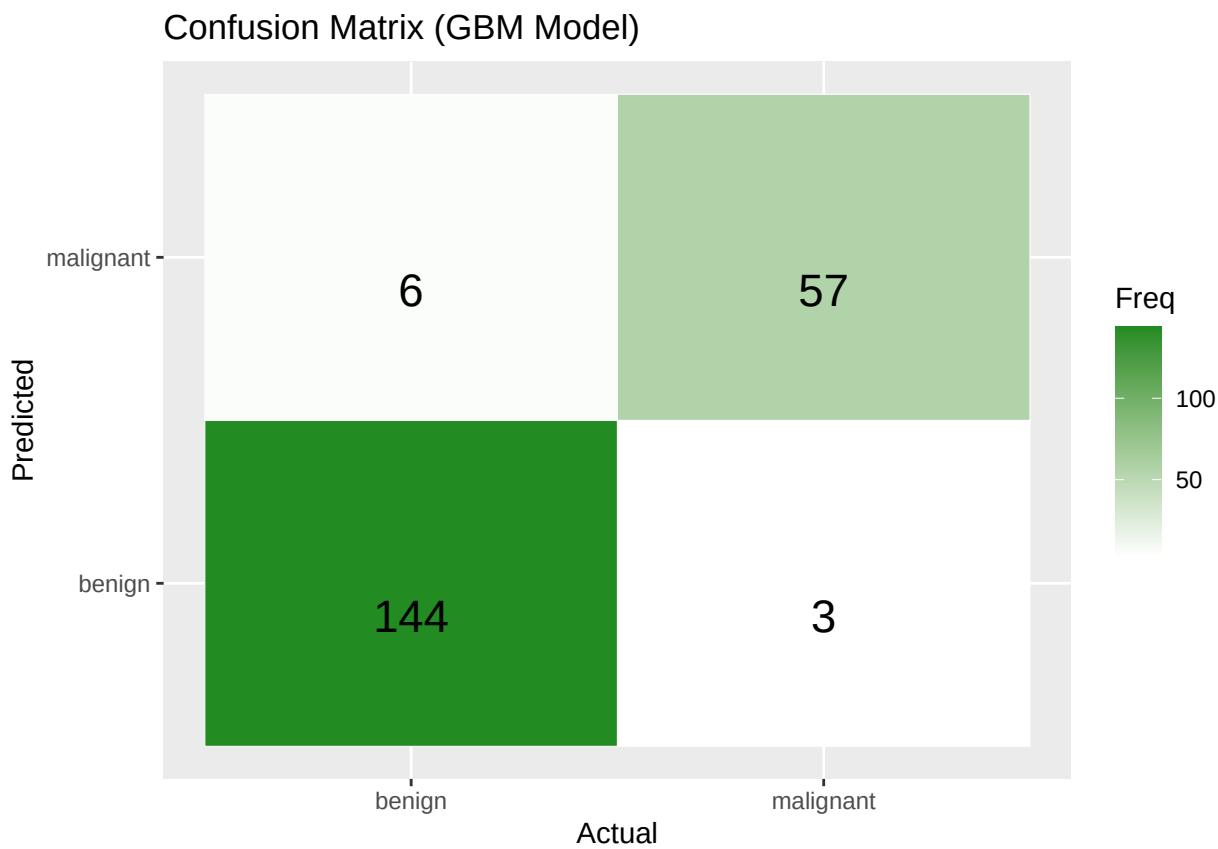
```r
table(rf_pred)
```

```
## rf_pred
##    benign malignant
##       150        60
```

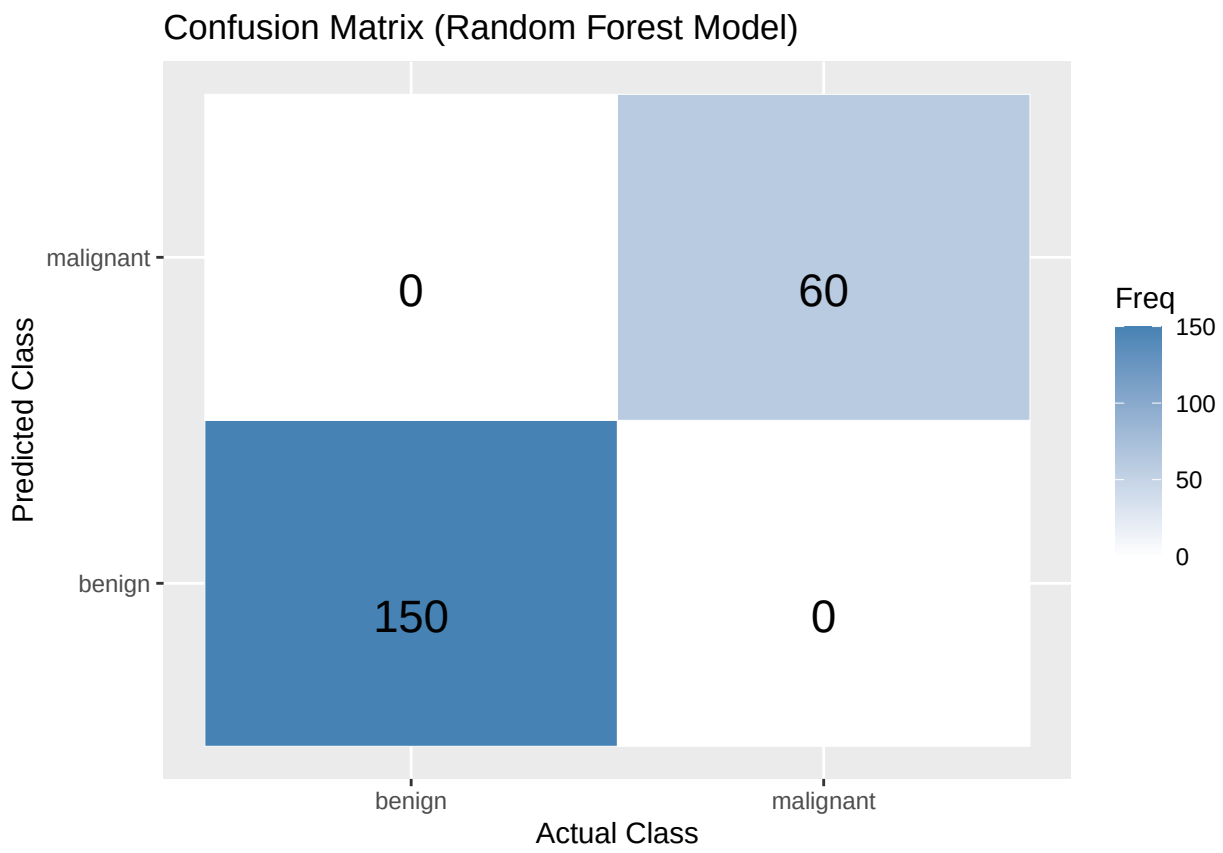
```r
#Generate confusion matrix
cm <- confusionMatrix(svm_pred, test$Class, positive = "malignant")

# Extract confusion matrix table
cm_df <- as.data.frame(cm$table)
names(cm_df) <- c("Prediction", "Reference", "Freq")

# Plot heatmap
ggplot(cm_df, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), vjust = 1.5, color = "black", size = 6) +
  scale_fill_gradient(low = "white", high = "steelblue") +
  labs(title = "Confusion Matrix (SVM Model)", x = "Actual", y = "Predicted")
```




```r
#Generate confusion matrix
cm <- confusionMatrix(gbm_pred, test$Class, positive = "malignant")

# Extract confusion matrix table
cm_df <- as.data.frame(cm$table)
names(cm_df) <- c("Prediction", "Reference", "Freq")

# Plot heatmap
ggplot(cm_df, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), vjust = 1.5, color = "black", size = 6) +
  scale_fill_gradient(low = "white", high = "forestgreen") +
  labs(title = "Confusion Matrix (GBM Model)", x = "Actual", y = "Predicted")
```




```r
#Generate confusion matrix
cm_rf <- confusionMatrix(rf_pred, test$Class, positive = "malignant")

# Create confusion matrix heatmap
cm_rf_df <- as.data.frame(cm_rf$table)
names(cm_rf_df) <- c("Prediction", "Reference", "Freq")

# Plot heatmap
ggplot(cm_rf_df, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), vjust = 1.5, color = "black", size = 6) +
  scale_fill_gradient(low = "white", high = "steelblue") +
  labs(title = "Confusion Matrix (Random Forest Model)", x = "Actual Class", y = "Predicted Class")
```


Confusion Matrix (Random Forest Model)
Predicted Class
malignant
benign
0
60
150
0
benign
malignant
Actual Class
Freq
150
100
50
0